SQUAD 4 · SPRINT PRODUIT KOP

Schéma de la base et dictionnaire des données

Document de référence destiné à l'équipe Data Science : structure relationnelle, règles métier et limites d'analyse de la plateforme KOP.

Périmètre	**14 octobre 2026**
Socle	**PostgreSQL · Neon**
Source	**Base vivante, extraite par introspection**
Généré par	`npm run docs:pdf`

KOP — annuaire des artisans du bâtiment à Brazzaville.
Document produit par extraction automatique : toute évolution du schéma se répercute à la prochaine génération.

SOMMAIRE

Ce que contient ce document

1. **Objet et périmètre**
ce que le document couvre, et ce qu'il ne couvre pas

2. **Architecture en une page**
les trois ensembles de tables et la règle de publication

3. **Planche 1 — Cœur du modèle**
comptes, profils et référentiels

4. **Planche 2 — Retours et modération**
avis, contacts et signalements

5. **Dictionnaire des données**
une entrée par table : rôle, colonnes, règles

6. **Grain**
ce que compte exactement une ligne de chaque table

7. **Comportement des suppressions**
mesuré sur la base, chaque essai annulé

8. **Invariants**
ce qui est garanti en base et ce qui ne l'est pas

9. **Limites pour l'analyse**
trois manques structurels à connaître

10. **Requêtes de référence**
les jointures utiles, exécutées et vérifiées

CHAPITRE 1

Objet et périmètre

KOP met en relation des particuliers et des artisans du bâtiment à Brazzaville. Un artisan publie un profil, un client le trouve, le contacte et peut laisser un avis. Ce document décrit la base qui porte ces usages.

> **Les volumes présentés sont ceux d'un environnement de développement.** Au moment de l'extraction : 4 profils, 10 comptes, 0 avis, 0 signalement. Un modèle statistique construit sur ces volumes n'aurait aucun sens — ces données valident la chaîne de production, pas une distribution. Pour des volumes réels, interroger la base de production, qui répond au même schéma.
>
> Quatre tables sont à ignorer : `pgmigrations`, `session`, `rate_limits` et `playing_with_neon` — cette dernière est l'exemple créé par l'hébergeur, absent du code de l'application.

Ce que le document contient

La structure complète : tables, colonnes, types, clés primaires et étrangères, unicité, contraintes et index. Les règles métier qui ne figurent pas dans le schéma, parce qu'elles vivent dans une vue ou dans un service. Et les limites du modèle, c'est-à-dire ce qu'il ne permet pas de mesurer.

Comment il a été produit

Par introspection de la base vivante, et non par relecture des migrations. Les migrations s'empilent et se modifient : la vue `published_professionals` a été réécrite quatre fois et la table `zones` a reçu `city_id` plus tard. Lire les migrations reconstitue une intention, pas l'état réel.

VOLUMES RELEVÉS

Ce que contient la base aujourd'hui

Relevé au moment de l'extraction, tables techniques exclues.

TABLE	LIGNES
`trade_categories`	5
`trades`	4
`cities`	6
`zones`	9
`users`	10
`professionals`	4
`professional_zones`	11
`photos`	7
`reviews`	0
`contacts`	1

TABLE	LIGNES
account_reports	0
review_reports	0

CHAPITRE 2

Architecture en une page

Douze tables se répartissent en trois ensembles. Les deux planches qui suivent les détaillent.

Identité et offre

Qui existe, et ce qu'il propose.

`users` porte toutes les identités, tous rôles confondus. Un compte devient artisan en passant `role` à `professional`, ce qui crée une ligne dans `professionals`. La clé primaire de cette dernière étant `user_id`, un compte porte **au plus un** profil.

Référentiels

Peu de données, beaucoup de contraintes.

`trades` et `trade_categories` décrivent les métiers, `cities` et `zones` la géographie. Ces tables ne se modifient pas par accident : toute suppression d'une ligne référencée est refusée par la base (voir chapitre 7).

Retours et modération

Ce que les comptes se renvoient.

`reviews` note les artisans, `contacts` journalise les échanges, `account_reports` et `review_reports` portent les signalements. C'est l'ensemble le plus exploitable analytiquement, et le plus petit aujourd'hui.

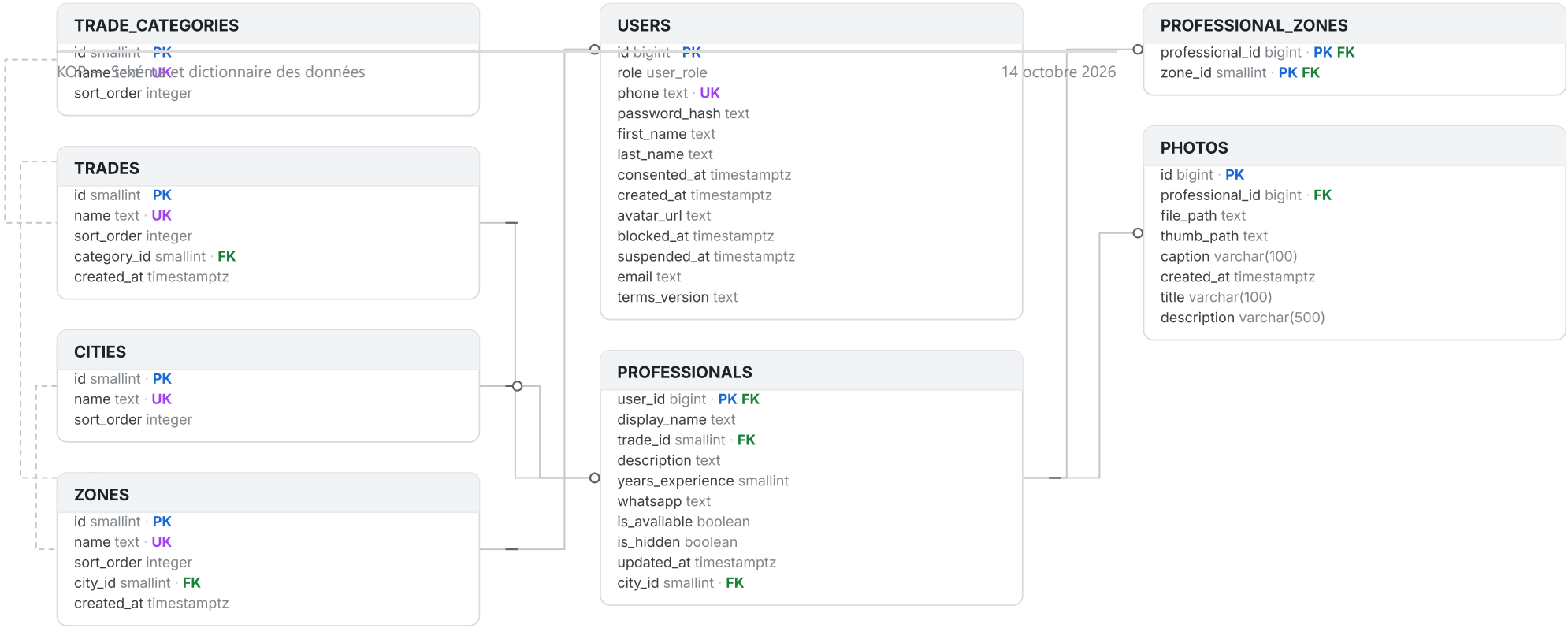
La règle de publication

Un profil n'est visible publiquement que s'il cumule **neuf conditions**, et cette logique vit dans la vue `published_professionals`, pas dans les tables : le profil n'est pas masqué, le compte est bien un artisan, il n'est ni bloqué ni suspendu, un nom affiché est renseigné, un métier est choisi, au moins une zone d'intervention est déclarée, un téléphone est présent, la description fait au moins 30 caractères, et au moins une photo a été ajoutée.

> **Compter les artisans avec `SELECT count(*) FROM professionals` est faux.** Il faut passer par la vue. L'écart se mesure sur les données actuelles : `professionals` compte 4 lignes, `published_professionals` n'en expose que **3**.

PLANCHE 1

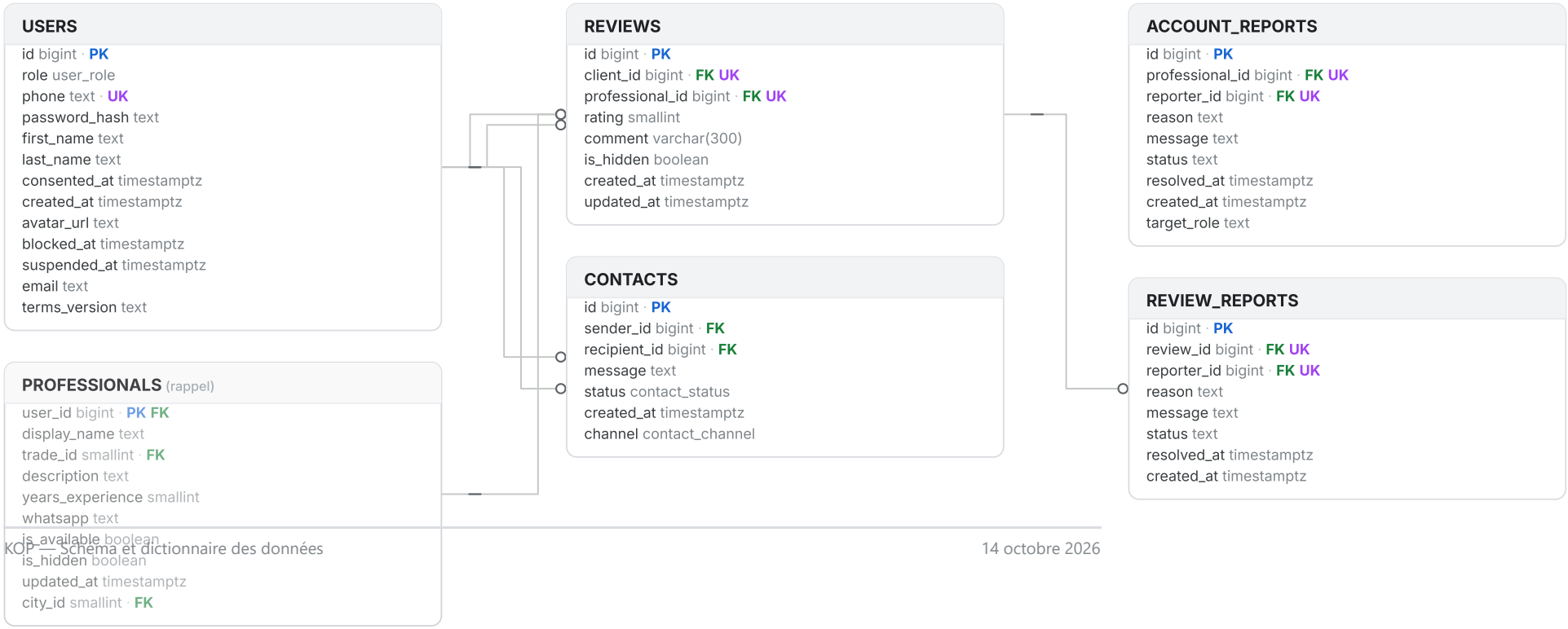
Cœur du modèle — comptes, profils, référentiels



PK clé primaire FK clé étrangère UK unicité Trait pointillé : lien à l'intérieur d'une même colonne

PLANCHE 2

Retours, contacts et modération



PK clé primaire **FK** clé étrangère **UK** unicité Trait pointillé : lien à l'intérieur d'une même colonne

CHAPITRE 5

Dictionnaire des données

Une entrée par table. La colonne « Rôle » dit ce que la table représente, la colonne « Clés » reprend les contraintes d'unicité et les clés étrangères.

`users`

Comptes — Toutes les identités de la plateforme, tous rôles confondus

L'authentification se fait par téléphone : `phone` est l'identifiant de connexion, pas une donnée de contact secondaire. `email` est facultatif et stocké normalisé en minuscules, avec une unicité insensible à la casse. `blocked_at` et `suspended_at` portent les sanctions : non nuls, ils retirent le compte de l'annuaire public sans jamais supprimer la ligne.

COLONNE	TYPE	CLÉS	GRAIN
id	bigint	PK	un compte
role	user_role	—	un compte
phone	text	UK	un compte
password_hash	text	—	un compte
first_name	text	—	un compte
last_name	text	—	un compte
consented_at	timestamptz	—	un compte
created_at	timestamptz	—	un compte
avatar_url	text	—	un compte
blocked_at	timestamptz	—	un compte
suspended_at	timestamptz	—	un compte
email	text	—	un compte
terms_version	text	—	un compte

Clés étrangères : aucune · Unicité : **phone**

`professionals`

Profils artisans — La vitrine publique d'un compte passé artisan

Une ligne au plus par compte, la clé primaire étant `user_id`. `is_available` décrit la disponibilité courante, ce n'est pas la publication : celle-ci est calculée par la vue `published_professionals`. `is_hidden` est un masquage volontaire du professionnel, distinct d'une suspension décidée par la modération.

COLONNE	TYPE	CLÉS	GRAIN
user_id	bigint	PK FK	un compte disposant d'un profil artisan
display_name	text	—	un compte disposant d'un profil artisan
trade_id	smallint	FK	un compte disposant d'un profil artisan
description	text	—	un compte disposant d'un profil artisan
years_experience	smallint	—	un compte disposant d'un profil artisan
whatsapp	text	—	un compte disposant d'un profil artisan
is_available	boolean	—	un compte disposant d'un profil artisan
is_hidden	boolean	—	un compte disposant d'un profil artisan
updated_at	timestamptz	—	un compte disposant d'un profil artisan
city_id	smallint	FK	un compte disposant d'un profil artisan

Clés étrangères : `user_id`, `trade_id`, `city_id`

`trades`

Métiers — Référentiel des métiers proposés

Quatre métiers au départ. Les identifiants sont figés et repris en dur dans la page d'accueil, qui les propose en accès direct.

COLONNE	TYPE	CLÉS	GRAIN
id	smallint	PK	un métier du référentiel
name	text	UK	un métier du référentiel
sort_order	integer	—	un métier du référentiel
category_id	smallint	FK	un métier du référentiel
created_at	timestamptz	—	un métier du référentiel

Clés étrangères : `category_id` · Unicité : `name`

`trade_categories`

Catégories de métiers — Regroupement des métiers pour la navigation

Référentiel prévu pour la navigation, non encore rattaché aux métiers en base.

COLONNE	TYPE	CLÉS	GRAIN
id	smallint	PK	une catégorie de métiers
name	text	UK	une catégorie de métiers
sort_order	integer	—	une catégorie de métiers

Clés étrangères : aucune · Unicité : `name`

`cities`

Villes — Référentiel géographique

Six villes congolaises. Une seule sert actuellement, Brazzaville.

COLONNE	TYPE	CLÉS	GRAIN
id	smallint	PK	une ville
name	text	UK	une ville
sort_order	integer	—	une ville

Clés étrangères : aucune · Unicité : `name`

`zones`

Quartiers — Zones d'intervention, rattachées à une ville

Neuf quartiers de Brazzaville. `city_id` est facultatif : une zone sans ville reste valide, elle est alors exclue des listes publiques.

COLONNE	TYPE	CLÉS	GRAIN
id	smallint	PK	un quartier
name	text	UK	un quartier
sort_order	integer	—	un quartier
city_id	smallint	FK	un quartier
created_at	timestamptz	—	un quartier

Clés étrangères : `city_id` · Unicité : `name`

`professional_zones`

Zones d'intervention — Association entre un artisan et un quartier

Table de jonction sans attribut propre. La clé primaire composite empêche deux fois la même zone pour un même artisan. Au moins une ligne est exigée pour qu'un profil soit publié.

COLONNE	TYPE	CLÉS	GRAIN
professional_id	bigint	PK FK	un couple (artisan, quartier)
zone_id	smallint	PK FK	un couple (artisan, quartier)

Clés étrangères : `professional_id`, `zone_id`

`photos`

Réalisations — Photos publiées par un artisan

Au moins une photo est exigée pour la publication. `file_path` et `thumb_path` sont des chemins relatifs : l'URL publique est reconstruite par l'application, vers Cloudinary en production.

COLONNE	TYPE	CLÉS	GRAIN
id	bigint	PK	une photo de réalisation
professional_id	bigint	FK	une photo de réalisation
file_path	text	—	une photo de réalisation
thumb_path	text	—	une photo de réalisation
caption	varchar(100)	—	une photo de réalisation
created_at	timestamptz	—	une photo de réalisation
title	varchar(100)	—	une photo de réalisation
description	varchar(500)	—	une photo de réalisation

Clés étrangères : `professional_id`

reviews

Avis — Note et commentaire laissés par un client

Un seul avis par client et par artisan, garantie par une contrainte d'unicité. La note va de 1 à 5. Aucun agrégat n'est stocké : la moyenne est recalculée à chaque lecture sur les seuls avis non masqués. Un avis se modifie et se supprime en place, sans conserve d'historique.

COLONNE	TYPE	CLÉS	GRAIN
id	bigint	PK	un avis — au plus un par (client, artisan)
client_id	bigint	FK UK	un avis — au plus un par (client, artisan)
professional_id	bigint	FK UK	un avis — au plus un par (client, artisan)
rating	smallint	—	un avis — au plus un par (client, artisan)
comment	varchar(300)	—	un avis — au plus un par (client, artisan)
is_hidden	boolean	—	un avis — au plus un par (client, artisan)
created_at	timestamptz	—	un avis — au plus un par (client, artisan)
updated_at	timestamptz	—	un avis — au plus un par (client, artisan)

Clés étrangères : `client_id`, `professional_id` · Unicité : `client_id`, `professional_id`

`contacts`

Prises de contact — Journal des échanges entre deux comptes

Le `channel` indique la forme de l'échange : `message`, `call` ou `whatsapp`. Un message n'est obligatoire que pour le canal `message`. `status` suit le cycle `new` puis `seen` puis `done` ; ouvrir sa liste de contacts fait passer les entrants en `seen`.

COLONNE	TYPE	CLÉS	GRAIN
id	bigint	PK	une prise de contact, dans un sens donné
sender_id	bigint	FK	une prise de contact, dans un sens donné
recipient_id	bigint	FK	une prise de contact, dans un sens donné
message	text	—	une prise de contact, dans un sens donné
status	contact_status	—	une prise de contact, dans un sens donné
created_at	timestamptz	—	une prise de contact, dans un sens donné
channel	contact_channel	—	une prise de contact, dans un sens donné

Clés étrangères : `sender_id`, `recipient_id`

`account_reports`

Signalements de compte — Signalements visant un client ou un artisan

La colonne `professional_id` porte la cible du signalement : sa clé étrangère pointe vers `users`, et `target_role` distingue un client d'un artisan. Le nom de colonne est donc trompeur, la contrainte s'appelle d'ailleurs `account_reports_target_fkey`. Un compte ne peut être signalé qu'une fois par le même auteur.

COLONNE	TYPE	CLÉS	GRAIN
id	bigint	PK	un signalement — au plus un par (cible, auteur)
professional_id	bigint	FK UK	un signalement — au plus un par (cible, auteur)
reporter_id	bigint	FK UK	un signalement — au plus un par (cible, auteur)
reason	text	—	un signalement — au plus un par (cible, auteur)
message	text	—	un signalement — au plus un par (cible, auteur)
status	text	—	un signalement — au plus un par (cible, auteur)
resolved_at	timestamptz	—	un signalement — au plus un par (cible, auteur)
created_at	timestamptz	—	un signalement — au plus un par (cible, auteur)
target_role	text	—	un signalement — au plus un par (cible, auteur)

Clés étrangères : `professional_id`, `reporter_id` · Unicité : `professional_id`, `reporter_id`

`review_reports`

Signalements d'avis — Signalements visant un avis, pas un compte

Mêmes règles qu'un signalement de compte, mais la cible est l'avis. Un avis signalé reste en base : la modération agit sur son contenu ou le masque.

COLONNE	TYPE	CLÉS	GRAIN
id	bigint	PK	un signalement d'avis — au plus un par (avis, auteur)
review_id	bigint	FK UK	un signalement d'avis — au plus un par (avis, auteur)
reporter_id	bigint	FK UK	un signalement d'avis — au plus un par (avis, auteur)
reason	text	—	un signalement d'avis — au plus un par (avis, auteur)
message	text	—	un signalement d'avis — au plus un par (avis, auteur)
status	text	—	un signalement d'avis — au plus un par (avis, auteur)
resolved_at	timestamptz	—	un signalement d'avis — au plus un par (avis, auteur)
created_at	timestamptz	—	un signalement d'avis — au plus un par (avis, auteur)

Clés étrangères : `review_id`, `reporter_id` · Unicité : `review_id`, `reporter_id`

CHAPITRE 6

Grain

Une ligne de, et le grain exact. Sans cette phrase, un `count(*)` n'a pas de sens interprétable.

TABLE	UNE LIGNE PAR…
`users`	un compte
`professionals`	un compte disposant d'un profil artisan
`trades`	un métier du référentiel
`trade_categories`	une catégorie de métiers
`cities`	une ville
`zones`	un quartier
`professional_zones`	un couple (artisan, quartier)
`photos`	une photo de réalisation
`reviews`	un avis — au plus un par (client, artisan)
`contacts`	une prise de contact, dans un sens donné
`account_reports`	un signalement — au plus un par (cible, auteur)
`review_reports`	un signalement d'avis — au plus un par (avis, auteur)

CHAPITRE 7

Comportement des suppressions

Mesuré sur la base : chaque essai a été annulé par `ROLLBACK` avant d'être consigné ici.

> **Toutes les clés étrangères sont non déferrable.** Sur une telle clé, `NO ACTION` se comporte comme `RESTRICT` : la suppression est refusée, elle n'est pas différée. Le `ON DELETE` affiché dans le dictionnaire indique donc `aucune`, mais la réalité est un blocage.

RELATION	DÉCLARÉ	COMPORTEMENT VÉRIFIÉ
`users → professionals`	CASCADE	Le profil, ses zones et ses photos partent.
`users → contacts`	CASCADE	Les prises de contact partent.
`users → account_reports`	CASCADE	Les signalements dont il est l'auteur partent.
`users → review_reports`	CASCADE	Les signalements d'avis dont il est l'auteur partent.
`reviews → professionals`	aucune	REFUSÉ — 23503. Un artisan ayant des avis ne peut pas être supprimé.
`professionals → photos`	CASCADE	Part avec le profil.
`professionals → professional_zones`	CASCADE	Part avec le profil.
`zones → professional_zones`	aucune	REFUSÉ — 23503. Un quartier référencé ne peut pas être supprimé.
`trades → professionals`	aucune	REFUSÉ — 23503. Un métier référencé ne peut pas être supprimé.
`cities → zones`	SET NULL	La zone survit, devient orpheline.
`cities → professionals`	SET NULL	Le profil survit sans ville.

Ce que cela change pour une analyse

Aucun avis ne peut devenir orphelin. Le blocage de clé étrangère empêche la suppression d'un artisan ayant des avis avant qu'elle ne rompe la référence : la jointure `reviews → professionals → users` ne perd donc aucune ligne par suppression de compte.

Mais le `CASCADE` est neutralisé par ce même blocage. Supprimer un compte cascade vers `professionals`, qui bute sur `reviews` : l'opération échoue. Pour retirer un compte, il faut d'abord supprimer ses avis, ses photos et ses zones.

CHAPITRE 8

Invariants

Une règle écrite dans la base vaut pour tous les écrivains : script, migration, console d'administration, ou le code qu'on a oublié. Les règles qui n'existent que dans l'application sont signalées comme telles.

INVARIANT	OÙ	DÉTAIL
Note entre 1 et 5	**Base**	CHECK sur reviews.rating
Description entre 30 et 500 caractères	**Base**	CHECK sur professionals.description
Expérience entre 0 et 60 ans	**Base**	CHECK sur professionals.years_experience
Motif de signalement dans une liste fermée	**Base**	CHECK sur les deux tables de signalement
Statut de signalement dans une liste fermée	**Base**	CHECK (pending, resolved, dismissed)
Un seul avis par (client, artisan)	**Base**	UNIQUE (client_id, professional_id)
Un seul signalement par (cible, auteur)	**Base**	UNIQUE sur les deux tables
Pas d'auto-signalement	**Base**	CHECK cible <> auteur
Pas de contact avec soi-même	**Base**	CHECK sender_id <> recipient_id
Nom affiché unique, insensible à la casse	**Base**	Index unique sur lower(btrim(...))
Un artisan ne s'avise pas lui-même	**Service**	reviews.service.js
Un administrateur ne pose pas d'avis	**Service**	reviews.service.js
Un profil n'est public que s'il est complet	**Vue**	published_professionals
Blocage à 3 signalements, suspension à 5	**Service**	Seuils configurables par variable d'environnement

CHAPITRE 9

Limites pour l’analyse

Trois manques structurels, à connaître avant de construire un indicateur.

professionals n’a pas de created_at

Seule la date de dernière modification existe. Impossible de dater l’arrivée d’un artisan ni de mesurer un délai de mise en ligne. Contournement : users.created_at donne la date de création du compte, identique ou antérieure.

L’historique des notes n’existe pas

Un avis se modifie et se supprime en place. Seule sa valeur courante est conservée, sans table d’audit. Une série « note moyenne dans le temps » sera fausse, et silencieusement fausse.

account_reports.professional_id est un piège

La colonne pointe vers users.id et sert de cible à un signalement de compte, client ou artisan. C’est target_role qui distingue les deux : une jointure sur professionals depuis cette colonne perd la moitié des lignes.

CHAPITRE 10

Requêtes de référence

Ces requêtes ont été exécutées sur la base avant d'être publiées.

Artisans publiés, avec métier, ville et zone

```
SELECT p.user_id, p.display_name, t.name AS trade,
       c.name AS ville, z.name AS zone
  FROM published_professionals p
  JOIN trades t ON t.id = p.trade_id
  JOIN professional_zones pz ON pz.professional_id = p.user_id
  JOIN zones z  ON z.id = pz.zone_id
  JOIN cities c ON c.id = z.city_id
 ORDER BY t.name;
```

Note moyenne par artisan, avis masqués exclus

```
SELECT p.user_id, p.display_name,
       ROUND(AVG(r.rating), 1) AS note_moyenne,
       COUNT(r.id)            AS nb_avis
  FROM published_professionals p
  LEFT JOIN reviews r
    ON r.professional_id = p.user_id AND NOT r.is_hidden
 GROUP BY p.user_id, p.display_name;
```

Comptes actifs, hors sanctions

```
SELECT id, role, created_at
  FROM users
 WHERE blocked_at IS NULL AND suspended_at IS NULL;
```

Comptes.signalements : cible et motif

```
SELECT r.created_at, r.reason, r.target_role,
       reporter.id AS auteur, cible.id AS cible
  FROM account_reports r
  JOIN users reporter ON reporter.id = r.reporter_id
  JOIN users cible    ON cible.id    = r.professional_id
 ORDER BY r.created_at DESC;
```

ANNEXE

Méthode de génération

Ce document ne s'écrit pas à la main.

ÉLÉMENT	SOURCE
Tables, colonnes, types, défauts	`information_schema, pg_attribute`
Clés primaires, unicité, clés étrangères	`pg_constraint`
Index	`pg_indexes`
Comportement des suppressions	`essais annulés par ROLLBACK`
Planches	`SVG déterministe, sans dépendance de rendu`
Contenu éditorial	`scripts/dictionary-content.mjs`

Évolutivité du schéma

31 migrations appliquées au moment de l'extraction. Le schéma évolue par migration SQL versionnée dans `server/db/migrations`, jamais par une modification directe en base. Pour régénérer : `npm run docs:pdf`.